Introduction to threads

04 November 2025 10:23

The concept of thread is not a new one: for sometimes, many operating systems have had libraries that provide the C programmer with a mechanism to create threads. Other languages, such as Ada have support for threads embedded into the language, much as support for threads is built into the Java language. Nonetheless, the topic of thread is usually considered a peripheral programming topic, one that's only needed in special programming cases.

With Java, things are different. It is impossible to write any but the simplest Java program without introducing the topic of threads. And the popularity of Java ensures that many developers who might never have considered learning about threading possibilities in a language like C or C++ need to become fluent in threaded programming.

Java Terms

Java: First is the term Java itself. As we know, Java started out as a programming language, and many people today think of Java as being simply a programming language. But Java is much more than just a programming language. It's also an API specification and a virtual machine specification. So when we say Java. We mean the entire Java platform. A programming language. An API and a virtual machine specification that taken together. Define an entire programming and runtime environment. Often we say Java. It's clear from the context that we are talking specifically about the programming language or parts of the Java API, or the virtual machine. The point to remember is that the threading features we discuss. Derive their properties from all the components of the Java platform, taken as a whole. While it's possible to take the Java programming language directly, compile it into assembly code, and run it outside of the virtual machine, such an executable may not necessarily behave the same as the programs.

Virtual machine, interpreters and browsers
The Java Virtual Machine is another term for the Java interpreter, which is the code that ultimately runs Java programs by interpreting the intermediate bytecode format of the Java programming language. The Java interpreter actually comes in three popular forms. The interpreter for developers (called Java) that runs program via a command line or a File Manager, the interpreter for end users (called jre) that is a subset of the developer environment and from the basis of(among other things) the Java plug-in, and the interpreter that is built into many popular web browsers such as:

- J**ava Web Start** is a better alternative to Java Applets for deploying Java applications, as it does not rely on browser plugins.
- Modern JDK versions no longer include the Java browser plugin, making Applet support obsolete in most environments.
- For secure and updated Java application deployment, transitioning to Java Web Start or standalone Java applications is recommended.

Programs, applications
This leads us to the term that we will use for things written in the Java language. Generally we call such entities programs. But there are two types of programs a typical Java programmer might write. Programs that can be run directly by the Java interpreter. And programs designed to be run by the Java enabled browsers. Fine.
The term application for standalone Java programs.

What exactly thread is a Thread?
The term thread is a shorthand, for a thread of control and a thread of control is, at its simplest, a section of code executed independently of other threads of control within a single program.

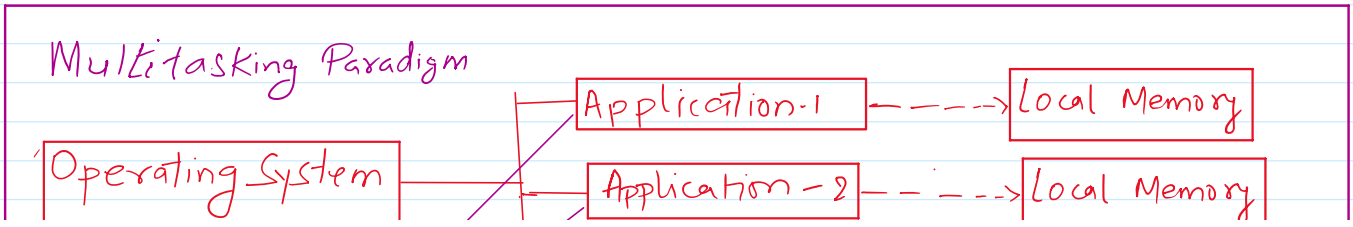
Thread of control
Threat of control sounds like complicated technical term, but it's really a simple concept: It is a path taken by a program during execution. This determines what code will be executed: does the if block get executed or does the else block. How many times does the while loop execute? If we were executing task from a "to do" list as much as computer executes an application what setup we perform and the order in which we perform them? Is our path of execution the result of our thread of control.

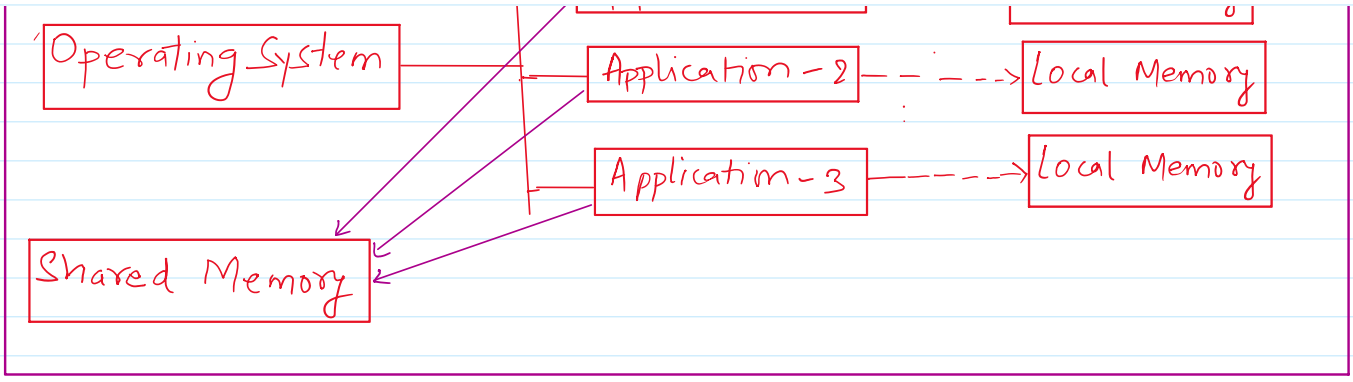
Having multiple threads of control is like executing task from two lists. We're still doing the task on each "to do" list in the correct order, but when we get bored with the task on one of the list, we switch list with the intention of returning at some future time to the first list at the exact point where we left off.

Overview of Multitasking
We all are familiar with the use of multitasking operating system to run multiple programs simultaneously. Each of these programs has at least one thread within it, so at the same level we are already comfortable with the notion of a thread in a single process. The single threaded process has the following properties, which as it turns out are shared by all threads in a program with multiple threads as well:

- The process begins execution at a well-known point. In programming languages like C and C++ (not to mention Java itself) the thread begins the execution at the first statement of the function or method called main()
- Execution of statement follows incompletely ordered pre-defined sequence for a given set of inputs. An individual process is a single minded in this regard. It simply executes the next statement in the program.
- While executing the process has access to certain data. In Java there are three types of data a process can access: Local variables are accessed from the thread's stack, instance variables are accessed through object references and static variables are accessed through class or object references.

Now consider what happens when you sit at your computer and start two single threaded programs, a text editor, say, and a File Manager. You now have two processes running on your computer. Each process has a single thread with the properties just outlined. Each process does not necessarily know about the other process. Although depending on the operating system running on your computer, there are several ways in which the processes can send each other various messages. A common behavior is that you can drag a file icon from the File Manager into the text editor in order to edit the file. Each process thus runs independently of the other, although they can cooperate if they so choose.

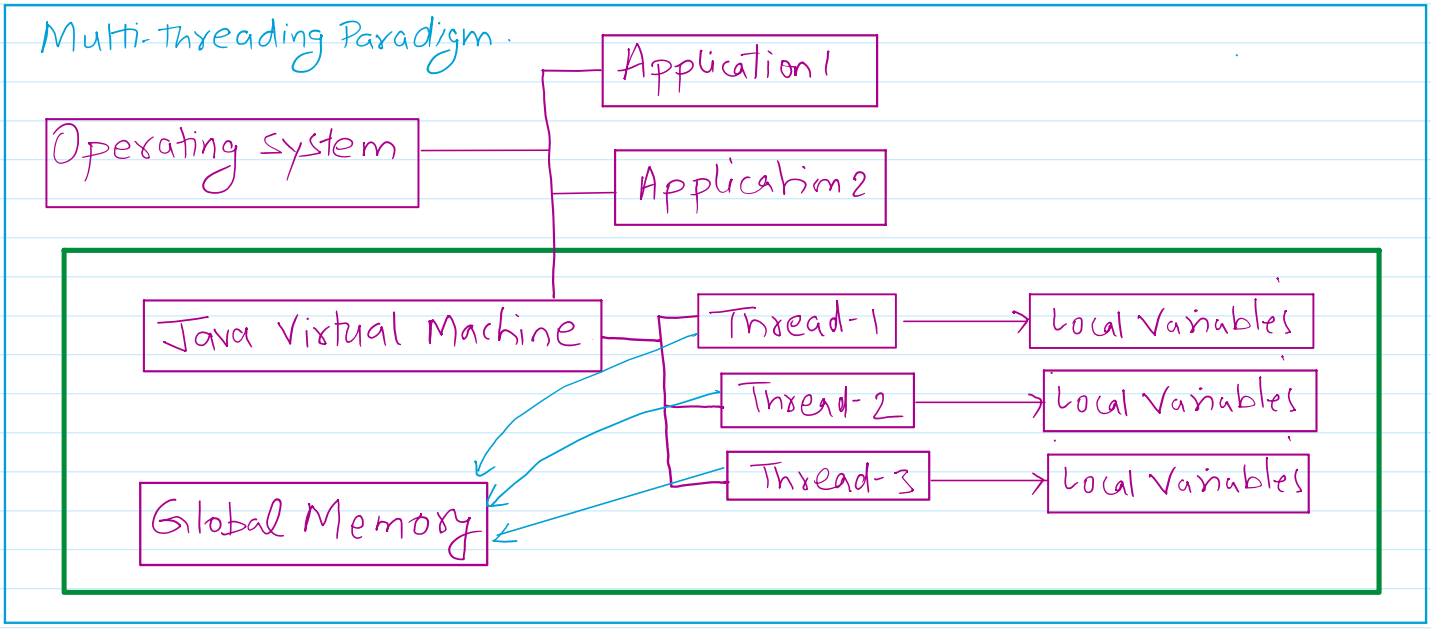




Processes in a multitasking environment.

From the point of view of a person using the computer, these processes often appear to execute simultaneously, although many variables can affect that appearance. These variables depend on the operating system: for example. A given operating system may not support multitasking at all, so that no two programs appear to execute simultaneously. Or the user may have decided that a particular process is more important than other processes and hence should always run, shutting out the other processes from running and again affecting the appearance of simultaneity.
Finally, the data contained within these two processes is by default separated: Each has its own stack for local variables, and each has its own data area for objects and other data elements. Under many operating system, the programmer can make arrangements so that the data objects reside in the memory that can be shared between the processes, allowing both processes to access them.

Overview of Multithreading
All of this leads us to a common analogy, Cologne. We can think of a thread just as we think of a process, and we can consider a program with multiple threads running within a single instance of a Java Virtual Machine, just as we consider multiple processes within an operating system.



Multi-tasking versus threading.

So it is that within a Java program multiple threads have these properties:

- Each thread begins execution at a predefined, well known location. For one of the thread in the program, that location is main () method; For the rest of the thread, it is a particular location the programmer decide on when the code is written.

- Each thread executes code from its starting location in an ordered predefined (for a given set of inputs) sequence. Threads are single minded in their purpose, always simply executing the next statement in the sequence.

- Each thread execute its code independently of the other threads in the program. If the threats choose to cooperate with each other, there are variety of mechanism we will explore that allow that cooperation. Exploiting those methods of cooperation is the reason why programming with threads is such a useful technique, but that cooperation is completely optional, much as the user is never required to drag a file from the File Manager into the text editor.
- The threads appear to have a certain degree of simultaneous execution. The degree of simultaneity depends on several factors. Programming decisions about the relative importance of a various threats as well as operating system support for various features. The potential for simultaneous execution is the key thing. You must keep in mind when threading your code.
- The thread have access to various types of data. At this point, the analogy to multiple processes breakdown somewhat depending on the type of data the Java program is attempting to access.

Each thread is separate so that local variables in the method that the thread is executing are separate for different threads. These local variables are completely private; There is no way for one thread to access the local variables of another thread. If two threads happen to execute the same method, each thread gets a separate copy of the local variables of that method. This is completely analogous to running two copies of the text editor: Each process would have separate copies of the local variable.

Objects and their instance variables on the other hand can be shared between threads in a Java program and sharing these objects between threads of Java program is much easier than sharing data objects between processes in most operating system. In fact, the ability to share data objects easily between threads is another reason why programming with threads is so useful. But Java threads cannot arbitrarily access each others data object: They need permission to access the object and one thread needs to pass the object reference to the other thread.

Static variables are the big exception to this analogy: They are automatically shared between all threads in the Java program.

Don't panic over this analogy: The fact that you will be programming with threads in Java doesn't mean you will necessarily be doing the system level type of programming you need to perform if you were writing this multitasking operating system responsible for running multiple programs. The Java thread API is designed to be simple and requires little specialized skill for most common task.

Why Threads?

The notion of threading is so ingrained in Java that it's almost impossible to write even the simplest program in Java without creating and using threads. And many of the classes in

the Java API are already threaded so that. Often you are using multiple threads without realizing it.

Historically, threading was first exploited to make certain programs easier to write Cologne. If a program can be split into two separate tasks, it's often easier to program the algorithm as separate task or threads. Programs that fall into this category are typically specialized and deal with multiple independent tasks.

The relative rareness of these type of programs, making thread Ding in this category a specialized skill. Often these programs were written as a separate processes using operating system dependent communication tool such as signals and shared memory spaces to communicate between processes. This approach increased system complexity.

The popularity of threading increased when graphical interfaces become the standard for desktop computers because the threading system allows the user to perceive better program performance. The introduction of threads into these platforms didn't make the programs any faster, but it did create an illusion of faster performance for the user, who now had a dedicate thread to service input or display output.

Recently there's been a flurry of activity regarding a new use of threaded programs: To exploit the growing number of computers that have multiple processors. Programs that require a lot of CPU processing are natural candidates for this category, since a calculation that requires one hour on a single processor machine could (at least theoretically) run in a half an hour on a two processor machine or 15 minutes on a 4 processor machine. All that is required is that the program be written to use multiple threads to perform the calculation.

While computers with multiple processors have been around for a long time, we are now seeing these machines become cheap enough to be very widely available. The advent of less expensive machine with multiple processors and of operating system that provide programs with red libraries to exploit those processors has made threaded programming a hot topic as developers move to extract every benefit from these new machines. Until Java, much of the interest in threading centered around using threads to take advantage of multiple processors on a single machine.

However, threading in Java often has nothing at all to do with multiprocessors machines and their capabilities. In fact, the first Java virtual machines were unable to take the advantages of multiple processors on a machine and many implementations of the virtual machines. Still follow that model. However. There are also implementation of the virtual machine that to take advantages of the multiple processors that the computer may have. A correctly written program running in one of those virtual machines on a computer with two processors may indeed take roughly half the time to execute that it would take on a computer with a single processor. If you are looking to use Java to have your program scale to many processor, that is indeed possible when you use the correct virtual machine. However, even if your Java program is destined to run on a machine with a single CPU, threading is still very important.

The major reason threading is so important in Java is that threading Java has no concept of asynchronous behavior. This means that many of the programming techniques you have become accustomed to using in a typical programs are not applicable in Java; Instead you must learn a new repertoire of threading techniques to handle these cases of asynchronous behavior.

This is not to say there aren't other times when threats are handy programming technique in Java; Certainly it's easy to use Java for a program that implements an algorithm that

naturally lends itself to threading. And many Java programs implement multiple independent behaviors. Some of the circumstances in which Java threads are required component of the program due to the need for a synchronous behavior or to the elegance that threading lends to the problem.

Nonblocking I/O

In Java, as in most programming languages, when you try to get input from the user, you execute read() method specifying the users terminal (System.in in Java). When the program executes the read() method, the program will typically wait until the user types at least one character before it continues and executes the next statement. This type of I/O is called blocking I/O: The program blocks until some data is available to satisfy the read() method.

In this type of behavior is often undesirable. If you are reading data from network socket, that data is often not available when you want to read it: The data may have been delayed in transit over the network, or you may be reading from the network server that sends data only periodically. If the program blocks when it tries to read from the socket, then it's unable to do anything else until the data is actually available.

If the program has a user interface that contains a button and the user presses the button while the program is executing the read () method, nothing will happen: The program will be unable to process the mouse events and execute the event processing method associated with the button. This can be very frustrating for the user who thinks the program has hung.

Traditionally, there are three techniques to cope with this situation:

I/O multiplexing: Developers often take all input sources and use a system call like select () to notify them when data is available from a particular source. This allows input to be handled much like an event from the user (In fact, many graphical toolkits use this method transparently to the user, who simply registers a callback function that is called when the data is available from the particular source).

Polling: Polling allows developer to test if data is available from a particular source. If that is available, the data can be read and processed; If it is not, the program can perform another task. Polling can be done either explicitly with a system call like pole () Or in some system, making the read() function return and indication that no data is immediately available.

Signals: A file descriptor representing an input source can often be set so that an asynchronous signal is delivered to the program when data is available on that input source. This signal interrupts the program which processes the data and then returns to whatever task it had been doing.

In Java, none of these techniques is directly available. There is a limited support for polling via the available() method of the FilterInputStream class, But this method does not have the rich semantics that polling typically has in most operating systems. To compensate for the lack of these features, a Java developer must set up a separate thread to read the data. This separate thread can block when the data isn't available, and the other threads in the Java program can process events from the user or perform other tasks.

While this issue of blocking IO can conceivably occur with any data source, it occurs most frequently with network sockets. If you're used to programming sockets, you have probably used one of these techniques to read from a socket, but perhaps not to write to 1. Many developers use two programming on a local area network are widely aware that writing to a socket may block, but it's a possibility that many of them ignore because it can only happen under certain circumstances, such as a backlog in getting data on to the network. This backlog rarely happens on a fast local area network, but if you are using Java to

program sockets over the Internet. The chances of this backlog happening? Are greatly increase; Hence the chances of blocking while attempting to write data onto the network is also increased. So in Java you may need two threads to handle the socket: one to read from the socket and one to write to it.

Alarms and Timers

Traditional operating systems typically provide some sort of timer or alarm call: the program sets the timer and continues processing. When the timer expires, the program receives some sort of a synchronous signal that notifies the program of the timers expiration.

In Java, the programmer must set up a separate thread to simulate a timer. This thread can sleep for the duration of a specified time interval and then notify the other threads that the timer has expired.

Independent Tasks

A Java program is often called on to perform independent tasks. In the simplest case. A single application may perform two independent animation for a web page. A more complex program would be a calculation server that performs calculation on behalf of several clients simultaneously. In either case, while it is possible to write a single threaded program to perform the multiple tasks, it's easier and more elegant to place each task in its own thread.

Parallelizable Algorithms

With the advent of virtual machines that can use multiple CPUs simultaneously, Java has become a useful platform for developing programs that use algorithms that can be parallelized. Any program that contains a loop is a candidate for being parallelized. That is, running 1 iteration of a loop on one CPU while another iteration of loop is simultaneously running on another CPU. Dependencies between the data that each iteration of the loop needs may prohibit a particular loop from being parallelized, and there may be other reasons why a loop should not be parallelized. But for many programs with CPU intensive loops, parallelizing the loop will greatly speed up the execution of the program when it is running on a machine with multiple processors.

Many languages have compilers that supports automatic parallelization of loops. As yet, Java does not.